kafka-goose-game

Implementation Documentation

July 2026

Overview

This is the full, top-down description of how the project is built and *why* it is built that way. It combines the system description, the technical choices and their motivations, the design patterns applied, the anti-patterns deliberately avoided, and the decisions and issues encountered along the way.

What this project is

A multiplayer **Game of the Goose** (Gioco dell'Oca) implemented in **plain Java 21** with the raw `kafka-clients` library — no Spring, no Kafka Streams, no framework of any kind. It exists to learn two things at once:

1. **Kafka's core mechanics**, hands-on: topics, partitions, replication, consumer groups, offset management, replay, delivery semantics, failure modes — by using the low-level client APIs directly instead of behind a framework's abstraction.
2. **Modern Java (21)**: records, sealed interfaces, exhaustive pattern matching, virtual threads, text blocks, immutable collections — used as the *primary* design tools, not as decoration.

The result is a playable game: a 3-broker Kafka cluster in Docker, one authoritative server process, and any number of terminal clients that join, roll dice, and watch an ANSI board update in real time.

The one idea everything follows from

> ***The Kafka log is the only state. Everything else is a cache.***

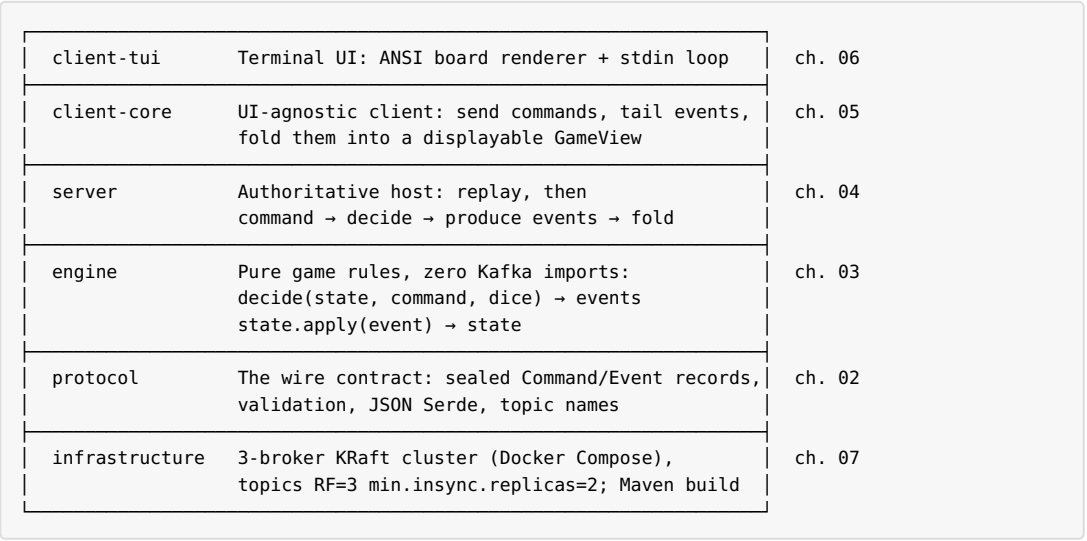
The system is **event-sourced** and **server-authoritative**:

- Clients never change state. They publish *intents* (`Command`s) to the `game.commands` topic.

- One server is the single authority. It turns commands into *facts* (`Event`s) via a pure rules engine and appends them to `game.events`.
- Every piece of in-memory state — the server's game map, every client's board view — is a **fold** (a left-reduce) over the event log. Kill any process, restart it, and it rebuilds itself by replaying the topic from the beginning.

Every design decision in the following chapters is a consequence of taking this idea seriously.

The layers, top-down



Dependencies point strictly downward, and deliberately *skip* layers where that keeps coupling low: `client-core` depends on `protocol` only — **not** on `engine` — so a client never carries game rules on its classpath (the motivation is examined in chapter 5).

Reading guide

Chapter	Contents
01 — Architecture	Event sourcing over Kafka: topics, keying, ordering, delivery semantics, the trust model, and the system-level patterns
02 — Protocol	The message hierarchy, validation-at-the-boundary, the hand-written JSON Serde, and its security posture
03 — Engine	The board rules, the decide/apply split, the goose-chain termination argument, and the deadlock rule amendment
04 — Server	Replay, the command loop, single-threaded ownership, at-least-once delivery, poison pills, graceful shutdown
05 — client-core	The client library: virtual-thread event loop, replay-on-start, the deliberately duplicated fold
06 — client-tui	The terminal UI: pure rendering, the serpentine board, I/O at the edge
07 — Infrastructure & build	The KRaft cluster, topic configuration, and the Maven multi-module setup
08 — Testing	The test strategy per layer, the deterministic E2E, and what live smoke-testing caught that unit tests could not
09 — Patterns & anti-patterns	The consolidated catalog: every pattern applied and every anti-pattern designed out, with pointers to where

Each chapter closes with a **Decisions** and an **Issues** section, the in-context version of the two project logs:

- `DECISIONS.md` — every non-obvious choice, with reasoning
- `ISSUES.md` — every problem hit, what was tried, what fixed it

For a hands-on introduction (quickstart, TUI commands, Kafka experiments to run against the live cluster), see the top-level README. The step-by-step build order the project followed is in `PLAN.md`.

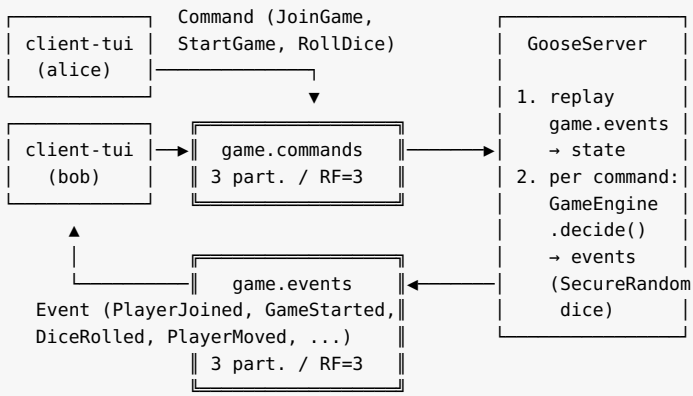
Fixed constraints the project was built under

These were decided up front (see `PLAN.md`) and never revisited:

Constraint	Value	Motivation
Language / stack	Plain Java 21 + `kafka-clients` 4.3.0	Learning goal: see Kafka without a framework in the way
Serialization	JSON via Jackson, hand-written `Serde`	Human-readable log (a learning feature in itself); no Schema Registry to operate
Cluster	3 KRaft brokers, topics RF=3, `min.insync.replicas=2`	Enough replication to *demonstrate* broker-loss survival, small enough for a laptop
Architecture	Event-sourced, server-authoritative, topics keyed by `gameId`	The pedagogical core of the project
UI	Terminal first, `client-core` kept UI-agnostic	A future web/desktop UI must be able to reuse the client layer unchanged
Tests	JUnit 5 everywhere; Testcontainers for the E2E	`mvn test` must never require Docker; `mvn verify` proves the real stack

01 — Architecture: Event Sourcing over Kafka

The shape of the system



Two topics, one direction each:

Topic	Written by	Read by	Meaning
`game.commands`	clients	the server	**Intents** — requests that may be rejected
`game.events`	the server only	the server (replay) and every client	**Facts** — things that irrevocably happened

This command/event split is the architecture. Everything below is about making each half honest.

Why event sourcing, and why here

Event sourcing is often overkill. It was chosen here because it is the single architecture that exercises the most Kafka concepts at once, on a domain small enough to hold in your head:

- **The log as source of truth** — Kafka's actual data model, not a queue metaphor. State is derived, disposable, and rebuildable (`replay`).
- **Ordering** — a game's correctness depends on event order; Kafka only orders within a partition, which forces you to understand keying.

- **Delivery semantics** — “did my event get written exactly once?” stops being an abstract FAQ and becomes a bug you can actually cause.
- **Multiple independent consumers** — server and N clients all read the same `game.events` with different group semantics and different needs.

A board game is an ideal domain for it: turns are inherently sequential events, state is small, and rules are pure logic — so the *infrastructure* concepts stay in focus.

Server-authoritative: the trust model

Only the server writes `game.events`, and only the server rolls dice (`SecureRandom`, chapter 4). Clients are untrusted by construction:

- A client cannot move a token — there is no command for it. It can only *ask* to join, start, or roll.
- An out-of-turn or otherwise invalid command produces **no events**; the server logs the rejection and moves on (chapter 3).
- A malicious or buggy client publishing garbage to `game.commands` is contained at two boundaries: the deserializer (payload cap, closed type set, field validation — chapter 2) and the engine (phase/turn checks).

The one thing the demo cluster does *not* enforce is broker-side write authorization — any process could write `game.events` directly. That is a conscious scoping decision: the cluster stays PLAINTEXT for learnability, and the README documents SASL/SCRAM + ACLs (clients write-only on commands, read-only on events) as the follow-up exercise that would close the gap.

Keying, partitions, and ordering

Both topics have **3 partitions** and every record is **keyed by `gameId`**. Consequences, in order of importance:

1. **Per-game total order.** All events of one game land in one partition, so every consumer sees that game’s history in exactly the order the server decided it. Cross-game order is not guaranteed — and doesn’t matter, because games are independent.

2. **Per-game single writer at scale.** Commands for one game also hash to one partition, so if the server were ever scaled to multiple instances in one consumer group, each game would still be processed by exactly one instance — no distributed locking needed. (The current server is deliberately single-instance; the caveat is recorded in chapter 4.)
3. **Parallelism headroom.** 3 partitions is enough to demonstrate points 1–2 and matches the 3 brokers; nothing more was needed.

The fold (`GameState.apply`, `GameView.apply`) filters or groups by `gameId`, so consumers reading the whole topic stay correct even though partitions interleave different games.

State is a fold — everywhere, identically

Three folds exist in the system, and their equivalence is a design invariant:

Fold	Where	Purpose
`GameState.apply(Event)`	engine, used by server	Authoritative state for rule decisions
`GameState.apply(Event)`	inside `GameEngine.decide`	The engine folds its *own* freshly-decided events before computing the next turn — decision logic and state logic share one source of truth
`GameView.apply(Event)`	client-core	Displayable state for UIs (adds a recent-event log)

`GameView` deliberately re-implements the fold rather than reusing `GameState` — the reasoning (dependency hygiene vs. code duplication, and why the wire protocol is the real contract) is examined in chapter 5.

The fold **trusts the log**. `apply` does not re-validate cross-event invariants (e.g. that a `TurnStarted` names a player who joined): those are guaranteed by the engine at decision time, and events are only ever produced by the server. Validating in both places would mean maintaining the rules twice — the exact duplicated-business-logic trap the code style rules warn about. What `apply`

does check is the per-event boundary: each record validates its own fields in its compact constructor, and `apply` rejects events addressed to a different `gameId`.

Delivery semantics: at-least-once, eyes open

The server implements **at-least-once** processing, and the limitation is documented rather than hidden:

- Producer: `acks=all` + `enable.idempotence=true` — an event acknowledged by the broker is on at least 2 of 3 replicas and was not duplicated by retries.
- The command consumer's offsets are committed **only after** every produced event is confirmed (`flush()` then `Future.get()` before `commitSync()`).

A crash between "events produced" and "offsets committed" therefore replays the commands from the last commit. For most commands the engine's rejection logic makes the replay a no-op (a second `JoinGame` for a joined player produces nothing), but a redelivered `RollDice` **rolls again** — dice are non-deterministic per state. True exactly-once would require Kafka transactions (produce + offset-commit atomically); that was deliberately left out of scope as the wrong complexity for a learning project, and the trade-off is recorded in `DECISIONS.md`. The important part pedagogically: the failure window is *understood and chosen*, not stumbled into.

Clients need no delivery guarantees at all: they never commit offsets, they replay from the beginning on every start, and folding is idempotent from a fixed starting state.

Replay: three consumers, three offset strategies

The same topic is read three different ways, which together cover most of Kafka's consumer-offset design space:

Consumer	Group	Offsets	Why
Server replay (startup)	none — manual `assign` + `seekToBeginning`	never committed	Replay must *always* read everything; group semantics (rebalancing, committed positions) would actively fight that
Server command loop	durable group `goose-server`	committed manually after produce-confirm	The one consumer whose position *is* meaningful state
Every client	throwaway group `goose-client-<uuid>`, `auto.offset.reset=earliest`	never committed	Each client start is a full replay; the group exists only because the consumer API requires one for `subscribe()`

Failure containment

- **Poison pills** (records that fail deserialization) are logged and skipped by seeking past the offending offset — both server and clients. A consumer loop must never die because of one bad record it will re-read forever.
- **Broker loss**: RF=3 with `min.insync.replicas=2` means any single broker can die without data loss or downtime — verified live by playing a complete game with one broker stopped (chapter 8). Losing a second broker makes `acks=all` writes fail — loudly, which is the durability contract working as intended.
- **Listener/UI failures** on the client are caught and logged so a rendering bug cannot stop the event stream (chapter 5).

Patterns applied at this level

- **Event Sourcing** — state as a fold over an append-only log.
- **CQRS in miniature** — commands (write intents) and events (read facts) on separate channels with separate types; no shared “message” superclass.
- **Single Writer** — one authority per game’s history (the server; per partition via keying).

- **Functional core, imperative shell** — pure `decide`/`apply` in the engine, all I/O pushed to the server and client edges (chapters 3–5).

Anti-patterns avoided at this level

- **Dual writes** — the server never writes state *and* events as separate operations that could diverge; the local state map is always derived from exactly the events that were produced.
- **Database-as-message-bus / shared mutable state** — processes communicate only through the two topics; no process reads another's memory.
- **Trusting the client** — no client-computed moves, no client-side dice.
- **Hidden delivery semantics** — the at-least-once window is documented at the exact code site that creates it, instead of pretending to exactly-once.

Decisions in this chapter

From `DECISIONS.md`: rejected commands produce no events; at-least-once over exactly-once; replay via manual assign; state is throwaway; single-instance assumption; topic names as shared constants in `protocol`.

Issues hit at this level

From `ISSUES.md`: #7 — the well+prison mutual-trap deadlock, which looked like an engine bug but was really an *architectural* lesson: with an append-only log you cannot "fix the data", only fix the rules going forward, and old logs must still fold correctly (see chapter 3).

02 — Protocol: the Wire Contract

The `protocol` module is the shared language of the system: the only code that both the server side (`engine`, `server`) and the client side (`client-core`, `client-tui`) depend on. It contains the message hierarchies, their validation, the JSON Serde, and the topic names — and nothing else. It has no game rules and no I/O beyond serialization.

The message hierarchies

Two sealed interfaces, one per topic:

`Command` — a player's intent (`Command.java`):

Record	Fields	Meaning
`JoinGame`	gameId, player	Ask to enter the lobby
`StartGame`	gameId, player	Ask to close the lobby and begin
`RollDice`	gameId, player	Ask to roll on one's own turn

`Event` — a fact decided by the server (`Event.java`):

Record	Key fields	Meaning
`PlayerJoined`	player	Lobby entry accepted
`GameStarted`	players (turn order), firstPlayer	Lobby closed, play began
`TurnStarted`	player	Whose turn it now is
`DiceRolled`	player, die1, die2	The server's roll (informational — the moves carry the state change)
`PlayerMoved`	player, from, to, `MoveReason`	One movement segment; a single roll can emit several (goose chains, bounce)
`PlayerStuck`	player, square	Trapped on inn 19 / well 31 / prison 52
`PlayerFreed`	player	No longer trapped
`GameWon`	player	Landed exactly on 63; terminal

Every event also carries `gameId` and an `Instant timestamp`. `MoveReason` (`NORMAL`, `GOOSE`, `BRIDGE`, `BOUNCE`, `MAZE`, `DEATH`) makes each movement segment self-describing, so clients can render *why* a token moved without knowing any board rules.

Why records + sealed interfaces

- **Records** give value semantics (`equals`/`hashCode`/`toString`) correct by construction and immutability by default — exactly what a wire message is. The E2E test compares whole events with `assertEquals`; that works only because records define equality by content.
- **Sealed** hierarchies make the message set *closed and compiler-checked*. Every `switch` over `Command` or `Event` in the codebase is exhaustive with **no `default` branch** — adding a ninth event type refuses to compile until the engine fold, the client fold, and the TUI renderer all handle it. The compiler becomes the checklist for protocol evolution.

Granularity choice: many small facts over one big one

A single roll could have been one fat `TurnResolved` event carrying the dice, all movement, traps and the next player. Instead it is a *sequence* of small events (`DiceRolled`, `PlayerMoved`×n, `PlayerStuck`?, `PlayerFreed`?, `TurnStarted`/`GameWon`). Motivations:

- each event has one meaning and one consumer-side fold case — folds stay trivial;
- the log is readable move-by-move in a console consumer (a learning feature);
- animation-grade granularity is available to future UIs for free.

The cost — a turn is not atomic in the log — is harmless: partial sequences can only occur if the server dies mid-produce, and at-least-once redelivery regenerates the remainder of the turn (see chapter 4).

Validation at the boundary — and only there

Every field is validated in the record's **compact constructor**, with the shared checks centralized in package-private `Validation`:

- `gameId`: 1–64 chars of `[A-Za-z0-9_-]`

- `player`: 1–20 chars of `[A-Za-z0-9_-]`
- `players` list: non-empty, each name valid, **no duplicates**, defensively copied with `List.copyOf`
- dice 1–6, squares 0–63 (0 = off-board start), timestamps non-null
- `GameStarted` additionally checks `firstPlayer ∈ players` — an invariant *between* fields belongs to the record that holds both

The consequence is a strong system-wide guarantee: **an invalid message cannot exist as a Java object**. There is no code path — local construction, deserialization from the wire, test fixture — that can produce a `Command` or `Event` with a null field, an oversized name, or a die of 7, because Jackson itself goes through the canonical constructor of each record. Everything downstream (engine, server, clients) can consume messages without a single defensive null check, and does.

The tight `[A-Za-z0-9_-]` charset is a security decision, not a style one: names end up in log lines, ANSI terminal output, and shell-adjacent contexts (console consumers). Restricting the alphabet at the boundary kills injection concerns (log forging, ANSI escape smuggling, path/shell metacharacters) in one place instead of escaping in many.

The Serde: one class, three interfaces

`JsonSerde<T>` implements Kafka's `Serializer<T>`, `Deserializer<T>` **and** `Serde<T>` in a single stateless class (`serializer()`/`deserializer()` return `this`). One instance therefore serves plain producers/consumers today and Kafka Streams / `TopologyTestDriver` tomorrow. It is constructed per hierarchy: `new JsonSerde<>(Event.class)` or `new JsonSerde<>(Command.class)`.

Design points, each with its motivation:

Closed polymorphism — the security core

The JSON `"type"` discriminator comes from `@JsonTypeInfo(use = NAME)` + explicit `@JsonSubTypes` on the sealed interfaces. Jackson's **polymorphic default typing is never activated**. This is the difference between "the wire can name any class on the classpath" (the classic Jackson gadget-chain RCE vector) and "the

wire can name exactly these 11 record types and nothing else". The closed set also mirrors the sealed hierarchy: the compiler enforces it in Java, the annotation enforces it on the wire.

Payload cap before parsing

`MAX_PAYLOAD_BYTES = 10 KiB`, checked on the raw byte array *before* Jackson touches it. Real messages are a few hundred bytes; anything bigger is garbage or abuse, and rejecting it early bounds the memory a hostile producer can make the server parse.

Unknown fields are tolerated — explicitly

`FAIL_ON_UNKNOWN_PROPERTIES = false`, with a why-comment and a pinning test. This was a review finding (ISSUES.md #5): Jackson's default (`true`) was in force *by accident* — chosen by nobody, tested by nothing. In an event-sourced system the log lives indefinitely, so a newer producer must be able to add fields without poisoning every not-yet-upgraded consumer. The leniency is safe precisely because of the compact-constructor validation: an unknown *extra* field is ignored, but a missing *required* field still fails construction. Trade-off accepted: a misspelled field name is silently ignored.

Tombstone contract: null in, null out

Both `serialize(null)` and `deserialize(null)` return `null` — a deliberate, commented deviation from the project's "never return null" default, because Kafka's log-compaction tombstones *are* null payloads and a Serde must pass them through. Downstream code handles the null explicitly (the server treats a null command/event as "nothing to do").

Inline `Instant` handling

Timestamps serialize as ISO-8601 strings via a tiny inline `SimpleModule` (two anonymous classes) instead of pulling in the `jackson-datatype-jsr310` module — two small classes versus a whole dependency for one type. The deserializer guards the non-string-token case (`p.getValueAsString()` returns null for

e.g. `"timestamp": {}`) and raises a proper `ctx.wrongTokenException(...)` instead of a bare NPE — a review finding.

Poison-pill contract

Any failure to deserialize — malformed JSON, unknown `"type"`, validation failure inside a compact constructor, oversized payload — is wrapped in the protocol's own `DeserializationException` with the cause attached. The message uses `e.toString()` rather than `e.getMessage()` because the latter can be null, and the exception class name is exactly what identifies a poison pill in the logs (another review finding). Consumers catch Kafka's `RecordDeserializationException` wrapper and **seek past** the bad record — log-and-skip, never crash (chapter 4).

Topic names live here too

`Topics.COMMANDS` / `Topics.EVENTS` are constants in the protocol module because topic names *are* wire contract — server and clients must agree on them exactly like they agree on field names. Before this, each side had its own string literal (found in review). Known caveat, recorded in DECISIONS.md: `docker-compose.yml`'s `init-topics` service still spells the names in YAML, so renaming a topic touches `Topics.java` *and* the compose file.

Patterns applied

- **Algebraic data types** (sealed interface + records) as the message model.
- **Parse, don't validate** — deserialization *is* validation; downstream code receives only proven-valid values.
- **Fail fast at the boundary** with named-parameter error messages.
- **Explicit policy over silent default** — the unknown-fields behavior is set in code, explained in a comment, and pinned by a test, so the policy is a visible choice instead of an inherited accident.

Anti-patterns avoided

- **Jackson default typing** — the gadget-chain deserialization vector is designed out, not mitigated.
- **Stringly-typed messages** — no maps of strings, no “type” switches on raw JSON anywhere outside the Serde.
- **Defensive re-validation sprinkled downstream** — validation exists exactly once, at construction.
- **Null-returning APIs** — the two tombstone nulls are the single, commented, contract-required exception.
- **Swallowed causes** — every wrap passes the original exception along.

Decisions (from DECISIONS.md)

Unknown-field tolerance; tombstone nulls; tri-interface Serde; inline Instant; closed polymorphism; 10 KiB cap; poison-pill exception type; restricted name charset; topic constants in protocol.

Issues (from ISSUES.md)

#5 — Jackson’s unknown-field default silently in force; made explicit, commented, and pinned by a test.

03 — Engine: the Pure Rules Core

The `engine` module is the functional core of the system: the complete rules of the Game of the Goose with **zero Kafka imports** and zero I/O. It depends only on `protocol`. Everything in it is a pure function or an immutable value, which is what makes the rest of the system testable and the E2E test deterministic.

Three types, one seam:

Type	Role
`Board`	Movement rules only: what one dice roll does to a token position
`GameState`	Immutable snapshot; `apply(Event)` folds one event into the next state
`GameEngine`	The rule book: `decide(state, command, dice)` → the events the command causes
`DiceRoller`	One-method interface — the seam through which tests inject scripts and the server injects `SecureRandom`

The decide / apply split

The engine's central design is the separation of the two halves of event sourcing:

```
decide :  GameState × Command × DiceRoller  →  List<Event>     (may reject: empty list)
apply  :  GameState × Event                 →  GameState        (total: never rejects)
```

- **`decide` is where rules live.** It checks phase, turn ownership, player-count limits, duplicate names; it consults the board; it is the only code that creates events. It is pure: timestamps come from an injected `Clock`, dice from the injected `DiceRoller`, so the same inputs always produce the same events.
- **`apply` is where meaning lives.** It gives each event its effect on state and *trusts the log*: it does not re-check cross-event invariants, because events only ever come from `decide` via the server-authoritative topic. Re-validating in the fold would mean maintaining every rule twice — the duplicated-business-logic trap — and would make replay of historically valid logs

fragile as rules evolve (see the deadlock amendment below for exactly that scenario).

A subtle but load-bearing detail: **`decide` folds its own output through `apply` before computing whose turn is next**. After building the roll/move/trap events it does `after = state; for (e : events) after = after.apply(e);` and only then runs `advanceTurn(after, …)`. Turn logic therefore always operates on the state *as the log will record it* — decision logic and fold logic cannot drift apart, because the decision logic *uses* the fold.

Rejections produce no events

An invalid command (out-of-turn roll, join after start, duplicate name, start with one player, wrong gameId…) returns an **empty list**. No `CommandRejected` event, no exception. Motivations: the log stays a record of things that *happened*, rejection needs no replay semantics, and the server just logs it. The trade-off — clients get no feedback for rejected commands — is acceptable for a TUI that redraws on every accepted event, and is recorded in DECISIONS.md as the place to revisit if a future UI needs rejection feedback.

This is also what makes at-least-once redelivery mostly harmless: replaying an already-applied `JoinGame` or `StartGame` against the updated state simply produces nothing.

Board: movement as data

`Board.resolve(from, roll)` returns the full `List<Move>` a roll triggers — each `Move(from, to, reason)` is one segment, in order — and the caller reads the final resting square from the last element. Movement is *described*, not performed: the board mutates nothing and emits no events; `GameEngine` translates the segments 1:1 into `PlayerMoved` events. This is why one roll can produce several `PlayerMoved`s on the wire and why the TUI can animate or narrate every hop.

The rules encoded: bridge 6→12, maze 42→39, death 58→1, geese {5,9,14,18,23,27,32,36,41,45,50,54,59} repeat the movement, overshooting 63 bounces back by the excess, landing exactly on

63 wins. The inn (19), well (31) and prison (52) deliberately do **not** appear in `resolve`: they don't move the token — trapping is *turn* logic and lives in `GameEngine`. Each layer owns one concern.

The goose-chain termination argument

The naive rule "landing on a goose moves you forward by the roll again" does not terminate: from 59 with a roll of 8, the token overshoots, bounces back to 59 — a goose — hops *forward* 8 again, forever (ISSUES.md #1, caught at design time before a line of code).

The implemented rule: a goose hop repeats the movement **in the token's current direction**, and a bounce off 63 *reverses* the direction (the `Cursor(pos, step)` record carries a signed step; reflection sets it to `-|step|`). Termination is then provable:

- while moving forward, every hop strictly increases the position, so the chain either exits the goose set or reaches the reflection — at most once;
- after reflecting, every hop strictly decreases the position, and backward chains bottom out (on the classic layout they stop by square 42; a belt-and-braces clamp at 0 guarantees `resolve` can never emit an invalid square even on a hypothetical board).

Every chain is finite because a strictly monotonic sequence over a finite board with at most one direction flip must leave the goose set.

Validation at the boundary, again

`resolve` rejects `from` outside 0–63 and `roll` outside 1–12 with `IllegalArgumentException` (ISSUES.md #3, found by review): a roll of 0 on a goose square would repeat a zero-length hop forever. Before the fix this was unreachable through `GameEngine` — but only by accident of the `DiceRolled` constructor validating first. Public methods get their own guards; safety by call-site coincidence is not safety.

GameState: the immutable fold target

`GameState` is a record of records: `gameId`, `Phase` (`LOBBY`/`RUNNING`/`FINISHED`), `players` in join order (which *is* the turn order), `positions`, `stuck` (player → trapping square), and two `Optional`s (`currentPlayer`, `winner`) that make "no current player" and "no winner yet" unrepresentable as nulls. The compact constructor defensively copies every collection with `List.copyOf`/`Map.copyOf`, so no caller can mutate a state after the fact — the record is deeply immutable, and "modification" means building the next state (small private withers: `withPosition`, `withStuck`, …).

`apply` is an exhaustive pattern switch over the sealed `Event` hierarchy with no `default`: an added event type fails compilation here first. Two events are deliberate no-ops in terms of structure: `DiceRolled` (informational — the `PlayerMoved`s carry the change) folds to `this`.

GameEngine: turn flow and the traps

After a roll resolves, `decide` inspects the landing square:

1. **63** → `GameWon`, game over, no next turn.
2. **A trap** (inn/well/prison), *unless it would freeze the game* → `PlayerStuck`; if the trap is the well or prison and it already holds someone, that occupant is `PlayerFreed` — the traps **swap occupants**.
3. Otherwise the token just rests.

Then `advanceTurn` picks the next player, walking the rotation from the current one:

- players held in the **well/prison** are skipped (they wait for relief);
- a player at the **inn** encountered on the first pass is freed (`PlayerFreed`) but skipped once — the inn costs exactly one missed turn, implemented as a two-pass scan (pass 0 frees inn players, pass 1 finds the first eligible player). In a 2-player game the opponent naturally rolls twice in a row — faithful to the tabletop rule.

`GameStarted` carries `firstPlayer` and *implies* the first turn — no redundant `TurnStarted` at game start, so folds treat `GameStarted` as both “phase change” and “first turn assignment”.

The deadlock amendment

The classic rules contain a genuine deadlock: with two players, one in the well and the other landing in the prison, both are waiting for relief that can never come. This was documented during Step 4 as “faithful, if merciless” and accepted as improbable — and then it **happened in the very first live smoke game** (ISSUES.md #7): alice fell into the well, bob goose-hopped 45→52 into the prison, game frozen.

The fix (a user decision recorded in DECISIONS.md): **the last free player never gets trapped**. `freezesTheGame(state, lander, square)` waives the trap when (a) the square holds-until-replaced, (b) it is unoccupied (an occupied trap swaps, so someone stays free), and (c) every *other* player is already held in a well/prison — inn players don’t count, because the rotation frees them by itself. When waived, the lander simply rests on the square unharmed.

Two details are worth noting as event-sourcing lessons:

- The rule changes `decide` only. `apply` is untouched — old logs (including the frozen game) still fold identically. **You fix rules going forward; the log is immutable.**
- The all-players-trapped branch in `advanceTurn` (emit no turn at all) is now unreachable through `decide`, but is kept and commented as the defensive path for replaying logs that predate the rule.

Patterns applied

- **Functional core** — pure decision function; effects (time, randomness) injected as `Clock` and `DiceRoller`.
- **State machine** — `Phase` + exhaustive switches; illegal transitions are rejections, not exceptions.
- **Command → Event** (the write half of CQRS/ES) with the fold as the read half.
- **Make illegal states unrepresentable** — `Optional` over null, deep-immutable records, sealed hierarchies.

- **Strategy via functional interface** — `DiceRoller` is the whole test seam; no mocking framework exists in the project.

Anti-patterns avoided

- **Hidden global effects** — no `Instant.now()`, no `new Random()` inside logic; determinism is structural, not accidental.
- **Duplicated business rules** — rules exist once in `decide`; the fold trusts them (the *deliberate* fold duplication in client-core is a different trade-off, see chapter 5).
- **Unbounded loops on data-driven rules** — both potential infinite loops (naive goose, zero step) were eliminated by proof and boundary validation respectively.
- **Exception-driven control flow** — rejection is an empty list, not a thrown exception; exceptions are reserved for programming errors (nulls, invalid arguments).

Decisions (from DECISIONS.md)

No-event rejections; direction-preserving goose hops with the termination argument; inn costs one rotation; well/prison swap occupants + the 2026-07-03 last-free-player amendment; `GameStarted` implies the first turn; decide folds its own events; apply trusts the log; `Board.resolve` boundary validation; injected `Clock`/`DiceRoller`.

Issues (from ISSUES.md)

#1 — naive goose rule loops forever (design-time). **#3** — `Board.resolve(x, 0)` latent infinite loop (review). **#7** — the well+prison deadlock fired in the first live game; rule amendment + two pinning tests.

04 — Server: the Authoritative Host

`GooseServer` is the imperative shell around the engine: the single process allowed to write `game.events`. Its whole life is two phases inside one `run()` call:

```
run():
  1. replayEvents()      — rebuild Map<gameId, GameState> from game.events
  2. processCommands()   — loop: poll game.commands
                                → engine.decide()
                                → produce events to game.events (keyed by gameId)
                                → fold events into local state
                                → commit offsets (only after produce confirmed)
```

It depends on `engine` (and transitively `protocol`) and is ~280 lines including configuration — the point of the pure core is that the shell stays small.

Startup replay: state is throwaway

On every start the server rebuilds *all* state by reading `game.events` from the beginning. There is no snapshot, no database, no local file. Consequences:

- a crash can never leave state and log disagreeing — the log *is* the state;
- deploying a rules change is just a restart (the fold reinterprets nothing; `apply` semantics are stable — see the deadlock amendment in chapter 3);
- startup cost grows with log size, which is fine at game scale and is the textbook trade-off event sourcing makes (snapshots would be the next step, deliberately out of scope).

Replay uses a consumer with **no group**: manual `assign()` over all partitions + `seekToBeginning()`, auto-commit off, positions never committed. Group semantics — rebalancing, resuming from committed offsets — are features for *sharing progress*, and replay must do the opposite: always read everything, alone. Completion is detected by capturing `endOffsets()` first and polling until every partition's `position()` reaches it.

Before assigning, the server checks `partitionsFor(Topics.EVENTS)` and fails fast with a clear `IllegalStateException` ("is the cluster up and init-topics done?") if the topic is missing — auto-topic-creation is disabled clusterwide (chapter 7), so a missing topic means a half-started environment and the operator should know immediately, not after a silent empty replay.

The command loop: at-least-once, in the right order

Per poll batch:

1. Every command goes through `handleCommand`: get-or-create the game's state (`computeIfAbsent` — atomic get-or-insert instead of check-then-act), `engine.decide(state, command, dice)`.
2. Rejected commands (empty event list) are logged and dropped.
3. Accepted: every event is `producer.send(...)`, keyed by `event.gameId()` — the futures are collected. State is folded (`applyEvent`) immediately after; the local map is only ever fed by the exact events sent to the log.
4. After the batch: `producer.flush()`, then `Future.get()` on **every** pending send — surfacing any produce failure as an exception that kills the batch *before* step 5.
5. Only then `consumer.commitSync()`.

The ordering of 4 and 5 is the entire delivery-semantics story: offsets are never committed for commands whose events might not have reached the log. A crash anywhere before step 5 redelivers the commands; the engine's rejection logic absorbs most duplicates, a redelivered `RollDice` rolls fresh dice — the documented at-least-once window (chapter 1).

Producer config: `acks=all` + `enable.idempotence=true` — an acknowledged write is on ≥2 replicas (`min.insync.replicas=2`) and internal retries cannot duplicate it. Consumer config: durable group `goose-server`, auto-commit **off** (auto-commit would commit on a timer, i.e. potentially *before* the produce — the exact bug the manual ordering exists to prevent), `auto.offset.reset=earliest` so a brand-new group starts from the first command.

Threading: single-threaded by design

The thread that calls `run()` owns *everything*: both consumers (replay, then commands), the producer, and the `Map<String, GameState>`. There is no lock in the file because there is no sharing — **thread confinement** as the concurrency strategy, chosen deliberately over synchronization.

The one concession to the outside world is shutdown, and it follows Kafka's own rules:

- `KafkaConsumer` is *not* thread-safe; its single thread-safe method is `wakeup()`.
- `close()` (callable from any thread — the JVM shutdown hook, the E2E test) therefore only **signals**: it sets `volatile boolean running = false`, calls `wakeup()` on whichever consumer is active (an `AtomicReference` updated as `run()` moves between phases), and awaits a `CountDownLatch` with a 10-second bound.
- The run thread notices (`WakeupException` from `poll`, or the flag), exits its loop, and **closes its own resources** on the way out — try-with-resources in the same thread that opened them. A `started` flag keeps `close()` from waiting on a server that never ran; the latch counts down in `run()`'s `finally`, so `close()` is bounded even if the loop died of an exception.

`volatile` is used exactly for what it is safe for — single-variable visibility — never read-modify-write.

Dice: `SecureRandom`, server-side only

`main()` wires `DiceRoller` to a `SecureRandom` (behind the `RandomGenerator` interface). Clients cannot roll, and the server's rolls are not predictable from prior observations the way a seeded `java.util.Random` would be. For a game this is arguably ceremony — but the point of the project includes security-by-default habits, and the cost is one line. The E2E test injects a scripted roller through the same seam; `main` is the only place randomness exists.

Poison pills

Both consumers poll through a wrapper that catches Kafka's `RecordDeserializationException`, logs partition/offset/cause, and `seek(partition, offset + 1)` — skipping exactly the bad record. Without the seek, the next `poll()` re-reads the same record and the loop degenerates into a hot crash-retry cycle on one hostile byte array. A tombstone (null command/event) is likewise handled explicitly as "nothing to do". The server must outlive anything a client can put on `game.commands`.

The single-instance assumption

Commands are keyed by gameId, so a consumer group of N servers would shard games cleanly — each game processed by exactly one instance, no coordination. But each instance folds *only its own* partitions' events after startup replay, so its state for other instances' games would go stale (harmless — it never acts on them — but wasteful and confusing). Scaling out properly would mean replaying only assigned partitions and handling rebalances. Recorded in DECISIONS.md as an explicit, documented limit rather than an accidental one: today, run one server.

Patterns applied

- **Imperative shell** around the pure core — all Kafka, all threading, all logging lives here; zero game rules.
- **Thread confinement** as the concurrency model; signal-and-wait shutdown via the one thread-safe channel (`wakeup`).
- **Transactional outbox discipline in miniature** — produce-confirm before offset-commit is the same "don't acknowledge input until output is durable" ordering, achieved with Kafka primitives.
- **Fail fast on environment errors** (missing topic) vs. **contain data errors** (poison pills) — opposite strategies, each matched to its failure class.

Anti-patterns avoided

- **Auto-commit with side effects** — the default `enable.auto.commit=true` silently gives at-most-once for this workload; switched off and replaced with explicit ordering.
- **Fire-and-forget producing** — every `send` future is confirmed; a produce failure stops the batch instead of vanishing.
- **Shared mutable state across threads** — no state map behind a lock; confinement instead.
- **Crash-on-bad-input consumer loops** — the poison-pill seek.
- **Swallowed `InterruptedException`** — re-interrupt then propagate, in both `awaitAll` and `close`.
- **Resource ownership ambiguity** — the opener closes; `close()` from another thread never touches a Kafka client beyond `wakeup()`.

Decisions (from DECISIONS.md)

Single-threaded ownership; at-least-once (not exactly-once) with the reasoning; manual-assign replay; throwaway local state; single-instance assumption; deterministic E2E via the DiceRoller seam.

Issues (from ISSUES.md)

#4 — Testcontainers vs. Docker daemon 29: the shaded docker-java client spoke API 1.32, rejected by the daemon; env var and dependency overrides were dead ends (the client is *shaded*), fixed with the `api.version=1.44` system property baked into failsafe. **#6** — `kafka-clients` does not expose slf4j at compile scope; every logging module declares `slf4j-simple` itself.

05 — client-core: the UI-Agnostic Client Library

`client-core` is the seam that keeps UIs cheap: everything a client needs to participate in a game — sending commands, following events, folding them into displayable state — with **no console I/O and no UI assumptions**. A web or desktop UI would depend on this module exactly as the TUI does, implementing one interface.

Three types:

Type	Role
`GameClient`	Connection: command producer + event-loop consumer on a virtual thread
`GameView`	Immutable displayable state: the client-side fold, plus a recent-events log
`GameListener`	The UI contract: `onEvent(Event)`, `onViewUpdated(GameView)`

Dependency: **`protocol` only.** Not `engine` — deliberately.

The deliberately duplicated fold

`GameView.apply(Event)` re-implements the event fold instead of reusing the engine's `GameState`. This is the most debatable decision in the codebase, so the reasoning is spelled out:

Why duplicate?

- PLAN.md fixes the dependency graph: a client needs *no game rules* on its classpath. `GameState` lives in `engine`, and pulling `engine` in for its fold would drag the rule book (Board, GameEngine) into every future UI — clients that must never make rule decisions would have the means to.
- The two folds serve different masters and were already diverging: `GameView` additionally maintains `recentEvents` (a bounded display log) and will grow UI-facing conveniences that have no place in the engine.
- **The wire protocol — not a shared class — is the contract.** Server and client agree because they consume the same sealed `Event` hierarchy, and both folds are exhaustive switches

over it: a new event type breaks *both* compilations until both folds handle it. The shared protocol tests are the guard.

What it costs: the fold semantics exist twice (~100 lines), and a behavioral divergence between them would show as a client rendering a different board than the server believes. Accepted with eyes open and recorded in DECISIONS.md — this is the *coincidental-vs-shared* duplication judgment call: the two folds are intentionally allowed to evolve apart, which is exactly when extraction is the wrong move.

`GameView` itself follows the same value discipline as `GameState`: a record, deep-immutable via `List.copyOf`/`Map.copyOf` in the compact constructor, `Optional` for absence, exhaustive switch, private withers. `recentEvents` is capped at 10 (`RECENT_EVENTS_LIMIT`), oldest evicted first — a display log, not a history (the history is the topic).

Replay-on-start: the client keeps nothing

Every `GameClient` start creates a consumer with a **fresh group id** (`goose-client-<uuid>`), `auto.offset.reset=earliest`, auto-commit off, offsets never committed. So a client (re)started mid-game rebuilds its whole view by replaying `game.events` from the beginning — the "kill a client and watch the board reappear" demo is not a feature bolted on, it *is* the read model. The throwaway group exists only because `subscribe()` requires one; nothing is ever stored under it.

Events are filtered client-side by `gameId` (the topic is shared by all games) and tombstones are skipped.

Threading model

- **The event loop owns the consumer and the fold.** It runs on a **virtual thread** (`Thread.ofVirtual()`) — the poll loop is I/O-bound waiting, the canonical virtual-thread workload; a platform thread per client would be waste. The `view` field is `volatile` so any thread can read the latest snapshot via `view()` (safe because `GameView` is immutable — publication is the only concern).
- **Listener callbacks run on the event-loop thread, in log order.** The contract is documented on `GameListener`: a UI that

needs its own thread hands off itself. A listener exception is caught, logged and skipped — a UI rendering bug must not stop the event stream (`notifyListener`).

- **Command senders and `close()` may be any thread.** `KafkaProducer` is thread-safe by contract (the one Kafka client that is); shutdown follows the same signal-don't-touch protocol as the server: `volatile running` flag, `consumer.wakeup()` via `AtomicReference`, then `eventLoop.join(  CLOSE_TIMEOUT)` — Java 21's `join(Duration)` — logging a warning if the loop fails to stop in 10 s. The producer is closed by `close()` because `close()`'s thread owns it (it was created in the constructor, used via thread-safe API).

The `connect()` static factory

`GameClient`'s constructor is private and does **not** start the event loop; it creates the virtual thread *unstarted*. The public entry is:

```
        public static GameClient connect(String bootstrap, String gameId, GameListener listener) {
            var client = new GameClient(bootstrap, gameId, listener);
            client.eventLoop.start();
            return client;
        }
```

Motivation: starting a thread inside a constructor publishes `this` before construction completes (the classic *this-escape* — the thread could observe final fields half-initialized). The factory starts the thread only after the constructor has returned. This was a skill-review finding, fixed by restructuring rather than by suppression.

Sending commands

`send()` attaches a **completion callback** that logs any failure — the producer future must never be silently dropped, because for a fire-and-forget client that log line is the *only* signal a command was lost (review finding: the original code discarded the future). After each send the producer is `flush()`ed: at human-scale traffic, prompt delivery beats batching, and a player's `roll` should be on the wire before their finger leaves the key.

Client-side validation comes for free: `client.join("bad name!")` throws `IllegalArgumentException` from the `Command.JoinGame` compact constructor *before* anything touches Kafka — the protocol's parse-don't-validate design doing local duty.

Poison pills on the event stream are seek-past-skipped exactly as in the server.

Patterns applied

- **Ports and adapters** — `GameListener` is the port; the TUI (or any future UI) is an adapter; `client-core` knows none of them.
- **Read model / projection** (the query half of CQRS) — `GameView` projects the event stream into display shape.
- **Static factory over constructor** to prevent this-escape and give the operation a name (`connect`).
- **Virtual thread per long-lived I/O loop** — Java 21's intended use.
- **Immutable snapshot publication** — `volatile` reference to an immutable value; readers get consistency without locks.

Anti-patterns avoided

- **this-escape from a constructor** (thread started before construction completed) — restructured via the factory.
- **Dropped async results** — the producer callback.
- **UI exceptions killing the data pipeline** — listener isolation.
- **Client-side offset state** — nothing to migrate, nothing to corrupt; restart = replay.
- **Sharing a consumer across threads** — confinement + `wakeup()`, same as the server.

Decisions (from DECISIONS.md)

GameView fold duplication rationale; fresh-group replay-on-start; listener threading contract; `connect()` factory; per-command flush; recentEvents cap.

Issues (from ISSUES.md)

#6 — first compile failed: `kafka-clients` uses slf4j internally but does not expose it at compile scope; `slf4j-simple` declared explicitly (version managed by the parent POM).

06 — client-tui: the Terminal UI

`client-tui` is the smallest module and the proof that `client-core`'s seam works: a complete UI in two classes, depending on `client-core` only.

Class	Role
`BoardRenderer`	**Pure**: `GameView` → ANSI string. No I/O of any kind.
`Main`	All the I/O: args, stdin loop, printing, process lifecycle.

BoardRenderer: rendering as a pure function

`render(GameView)` returns the full screen as one string: the 63-square board, a status block (players, positions, whose turn, traps, winner banner), the recent-event log, and a legend. Keeping it pure has one motivation and one big payoff:

- **Motivation:** rendering logic is layout arithmetic — exactly the kind of code that harbors off-by-one bugs — and layout arithmetic is trivially unit-testable *if* no console is involved.
- **Payoff:** the tests strip ANSI codes with a regex and assert on plain text (`"55 "`, `"19I"`, `"12 ABCDEF"`, `"*** alice WINS! ***"`). Nine tests cover the serpentine layout, special-square markers, token placement, status, event lines, and the ANSI hygiene itself — with zero terminal automation.

The serpentine board

The classic board snakes. The grid is 7 rows × 9 columns, square 1 at the bottom-left, even rows (0-based, from the bottom) running left→right and odd rows right→left, so 63 ends in the top-left region:

```
55  56  57  58X 59* 60  61  62  63
54* 53  52P 51  50* 49  48  47  46
37  38  39  40  41* 42< 43  44  45*
36* 35  34  33  32* 31W 30  29  28
19I 20  21  22  23* 24  25  26  27*
18* 17  16  15  14* 13  12  11  10
 1   2   3   4   5*  6>  7   8   9
```

Each cell is 8 columns: number (2) + marker (1) + player initials + padding. Markers encode the special squares (`*` goose, `>` bridge, `<` maze, `X` death, `I` inn, `W` well, `P` prison), colored by class (traps red, jumps cyan, geese green); players get one ANSI color each by join position, their token being the bold uppercase initial of their name.

One found-in-review edge case is instructive: six players on one square emit 9 visible characters into the 8-wide cell, and the original padding computation called `" ".repeat(-1)` → `IllegalArgumentException` — a renderer crash on a *valid* game state. The fix is `Math.max(0, CELL_WIDTH - visible)` with a comment explaining the choice (give up one alignment column rather than throw) and a pinning test (`sixPlayersOnOneSquareStillRender`). Render code must be total over the state space it accepts.

Event narration

`eventLine(Event)` renders each event as one human sentence ("alice rolled 3+4 = 7", "bob moved 6 -> 12 (bridge)") — an exhaustive switch over the sealed hierarchy, so a new event type breaks this compile too. String formatting pins `Locale.ROOT` (both the `%2d` cells and the lowercased `MoveReason`): board output is machine-readable by the tests, and locale- dependent formatting in machine-read output is a classic latent bug (the Turkish-i problem).

Main: the imperative rim

`Main` owns everything impure, and nothing else:

- **Args** `[bootstrap [gameId [player]]]` with defaults `localhost:9092 game-1 <os-user>`; the OS username is sanitized to the protocol's `[A-Za-z0-9_-]{1,20}` shape (strip + truncate, fallback `"player"`) — the default must be *valid by construction* or the first suggested command would throw.
- **Stdin loop** on the main thread: `join [name]` / `start` / `roll` / `board` / `help` / `quit`. UTF-8 is pinned on the reader. A caught `IllegalArgumentException` (an invalid name typed at the prompt) prints `invalid: …` and continues — user errors are prompts, not stack traces.

- **Rendering trigger**: the `GameListener` callback (on the client's event loop thread) clears the screen and reprints on every view update. Both threads write to `System.out`; an occasionally interleaved prompt is the accepted cost of a plain-stdio TUI, documented in the class javadoc rather than "fixed" with a terminal library the project doesn't need.
- **Identity is a convention, not a session**: `join bob` switches which player subsequent `start`/`roll` act as (an `AtomicReference<String>`, since the listener thread reads it for rendering context). There is no authentication anywhere — a deliberate scope line: identity/authn is orthogonal to what the project teaches and is listed as future work with SASL in the README.

Two environmental details with reasons:

- `org.slf4j.simpleLogger.defaultLogLevel=warn` is set first thing in `main()`: kafka-clients logs INFO chattily, and INFO chatter scribbles over an ANSI-painted board.
- ANSI escapes appear in source **only as `\u001B` unicode escapes**, never as raw ESC bytes. Mid-project, generated sources briefly contained invisible 0x1B control characters — they survive copy-paste, break diffs subtly, and are a maintenance hazard; they were purged with `sed` and the convention was recorded in DECISIONS.md.

Blind rolls: an accidental design win

Because the server rejects out-of-turn `RollDice` with *no events and no error*, a client can be driven by the dumbest possible script — pipe `roll` every second — and the game still plays out correctly. This is what made the automated smoke games possible (two piped clients playing full games through the real cluster, chapter 8), and it fell out of the "rejections produce no events" decision rather than being designed for. It also promptly paid for itself by triggering the well+prison deadlock in the first live game.

Patterns applied

- **Functional core / imperative shell, in miniature** — the same split as engine/server, one layer up: `BoardRenderer` pure, `Main` impure.
- **Humble Object** — `Main` is deliberately too thin to need tests; all the logic that *could* be wrong lives in the testable renderer.
- **Exhaustive rendering** — the sealed hierarchy forces the UI to keep up with the protocol at compile time.

Anti-patterns avoided

- **Logic in the I/O layer** — no game or layout decisions in `Main`.
- **Partial functions in rendering** — the negative-repeat crash class, eliminated and pinned.
- **Locale-dependent machine output** — `Locale.ROOT` pinned.
- **Invisible control characters in source** — the `\u001B` convention.
- **Framework reflex** — no curses/JLine dependency for a problem plain stdio solves; the interleaved-prompt trade-off is documented instead.

Decisions (from DECISIONS.md)

Pure renderer; serpentine convention; `\u001B` escapes; kafka log level; blind-roll safety; `join` switches identity.

Issues (from ISSUES.md)

#7 was *found* through this module's smoke games (the deadlock — fixed in the engine). The overfull-cell crash and a test asserting a goose on a non-goose square (55) were review-cycle findings fixed before commit.

07 — Infrastructure & Build

The Kafka cluster

`docker-compose.yml` at the repo root defines the whole runtime environment: three brokers plus a one-shot topic initializer. Design choices, each with its reason:

KRaft, combined mode, three nodes

- **KRaft (no ZooKeeper)** — Kafka 4.x's native consensus; learning ZooKeeper operations in 2026 would be learning the past. All three nodes run in **combined controller+broker mode** (`KAFKA_PROCESS_ROLES: broker,controller`) with a controller quorum of all three — fine for dev, and one less container triple to operate. A production cluster would separate the roles; that distinction is exactly the kind of thing the setup makes visible.
- **Three brokers** is the minimum that makes replication *interesting*: RF=3 with `min.insync.replicas=2` survives one broker loss with full availability and fails writes loudly on the second loss — both behaviors were demonstrated live (chapter 8).
- **No volumes, on purpose** — `docker compose down` gives a factory-fresh cluster. For a learning project, disposability beats durability: every experiment starts from a known state.

Listeners: the dual-network reality

Each broker exposes two data listeners, because a Docker-hosted Kafka is reachable from two networks with two different addresses:

- `PLAINTEXT://kafka-N:29092` — for traffic *inside* the compose network (inter-broker, the init job);
- `PLAINTEXT_HOST://localhost:9092/9094/9096` — advertised to *host* processes (the server and clients run on the host).

Getting `advertised.listeners` right is the single most common Kafka-in- Docker stumbling block; the compose file documents both addresses in its header comment.

Topic hygiene

- `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"` — game topics are created explicitly by the `init-topics` one-shot service (3 partitions, RF=3, `min.insync.replicas=2`); a typo'd topic name should be an error, not a silently created 1-replica topic with defaults.
- The internal topics (`__consumer_offsets`, transaction state) are also pinned to RF=3 / min-ISR 2 — a broker loss must not take out *offset storage* either; forgetting this is a classic dev-compose gap.
- `init-topics` waits on all three brokers' **healthchecks** (a `kafka-broker-api-versions.sh` probe), creates both topics with `--if-not-exists` (idempotent re-up), describes them for the log, and exits 0 — Compose's `depends_on: condition: service_healthy` as the ordering mechanism, no sleep loops.

Known caveat (DECISIONS.md): topic names appear both in `Topics.java` (the code contract) and in this YAML — a rename touches both.

The Maven build

Parent POM + five modules (`protocol` ← `engine` ← `server`; `protocol` ← `client-core` ← `client-tui`). The parent does three jobs:

1. **Version pinning, once.** All library versions live in `<dependencyManagement>` (kafka-clients 4.3.0, jackson-databind 2.19.0, slf4j 2.0.17, JUnit BOM 5.12.2, Testcontainers BOM 1.21.3 — all verified current-stable on Maven Central at project start, deliberately skipping alpha/milestone lines). Modules declare dependencies without versions; there is exactly one place a version can be wrong.
2. **Java 21 via `maven.compiler.release`** — release, not source/target, so the compiler also checks API usage against the JDK 21 platform.
3. **Plugin management**: compiler 3.14.0, surefire/failsafe 3.5.3, exec 3.5.0 — pinned so builds don't drift with Maven defaults.

Module-level choices:

- **Unit tests vs. integration tests are split by plugin**: surefire runs `*Test` at the `test` phase everywhere; **failsafe is activated only in `server`** and runs `*IT` at `verify`. Consequence: `mvn test` never needs Docker; `mvn verify` runs the Testcontainers E2E. The failsafe config also carries the `api.version=1.44` system property — the workaround for the shaded docker-java client vs. Docker daemon ≥29 (ISSUES.md #4), commented with its removal condition.
- **`slf4j-simple` is declared per logging module** (server, client-core, client-tui): kafka-clients logs through the slf4j API but does not expose it at compile scope (ISSUES.md #6).
- **exec-maven-plugin** is configured with a `mainClass` in `client-tui` (so `mvn -pl client-tui exec:java` just works); the server is launched with an explicit `-Dexec.mainClass=com.goosegame.server.GooseServer`.

The reactor gotcha (worth knowing cold)

`mvn -pl engine test` fails on a fresh checkout: Maven tries to resolve the `protocol` SNAPSHOT from the repository, where it has never been installed. Two correct invocations (ISSUES.md #2):

- `mvn -pl <module> -am test` — `-am` ("also make") builds the in-reactor dependencies too;
- for `exec:java` runs, `mvn -q -DskipTests install` once, so sibling SNAPSHOTs exist in the local repository.

Patterns applied

- **Infrastructure as (reviewed) code** — the compose file went through the same review workflow as Java.
- **Fail-fast environments** — health-checked startup ordering, no auto-created topics, disposable state.
- **Single source of truth for versions** — dependencyManagement + BOMs.

Anti-patterns avoided

- **SNAPSHOT/latest dependencies** — every version pinned, including plugins.

- **Auto-topic-creation in a replicated cluster** — off.
- **Under-replicated internal topics** — offsets/transactions at RF=3.
- **Sleep-based container orchestration** — healthchecks + `depends_on` conditions.

Decisions (from DECISIONS.md)

Failsafe only in `server`; topic names duplicated in YAML (caveat); the build/workflow entries.

Issues (from ISSUES.md)

#2 — the `-am` reactor gotcha. **#4** — the Testcontainers/Docker-29 API-version saga (env var ignored, dependency override inert because the client is shaded, fixed via the `api.version` system property). **#6** — slf4j not transitively visible at compile scope.

08 — Testing Strategy

The test suite is shaped like the architecture: the purer the layer, the more tests it gets and the cheaper they are. 105 tests total, no mocking framework anywhere — every seam that needs faking (`DiceRoller`, `Clock`, `GameListener`) is a small interface a test can implement in one line.

Module	Tests	Kind	What they prove
protocol	46	unit	Round-trip of every message type; rejection of malformed / unknown-type / oversized / non-string-timestamp payloads; unknown-*field* tolerance; tombstone nulls; validation rules incl. duplicates
engine	39	unit	Every board rule; every rejection; trap/free/turn flow; the deadlock amendment; full scripted games with fixed dice
server	1	E2E (Testcontainers)	The entire stack against a real broker
client-core	10	unit	The view fold over scripted event sequences (replay logic)
client-tui	9	unit	Board layout, markers, tokens, status, winner banner, event narration, ANSI hygiene, the overfull-cell edge

Why the pyramid is this shape

- **Protocol and engine carry the weight** because they are pure: tests are milliseconds, need no infrastructure, and can enumerate rules exhaustively. A scripted full game in

`GameEngineTest` folds every event through `GameState` exactly as the server would — so most "integration" behavior is already proven below the integration layer.

- **The server gets one test, but it is the whole system.** Everything the server adds — Kafka wiring, replay, produce-confirm-commit ordering, shutdown — is only meaningful against a real broker. Unit-testing it would mean mocking `KafkaConsumer`, which tests the mock, not the contract.
- **`mvn test` never needs Docker** — the E2E is an `*IT` under failsafe at `verify` (chapter 7). Fast feedback for logic, full proof on demand.

The deterministic E2E

`GooseServerIT` starts a throwaway `apache/kafka:4.3.0` container (Testcontainers), creates the topics via the Admin API, runs the real `GooseServer.run()` on a **virtual thread**, and then plays a two-player game *entirely from outside* — producing commands and consuming events over the network, with no access to server internals.

Determinism is designed, not hoped for:

- **Scripted dice** injected through the same `DiceRoller` seam production uses (a queue: `queue::remove` — exhaustion throws, so a script/logic mismatch fails loudly instead of rolling garbage). The script is chosen so the game exercises a goose hop, the bridge, and an exact landing on 63.
- **The driver is reactive, not timed.** It answers each `GameStarted`/`TurnStarted` event with exactly one `RollDice` for the announced player, until `GameWon`. There are no sleeps and no offset arithmetic — the event stream itself is the synchronization. A 90-second deadline exists only as a failure backstop, and its failure message dumps the event history collected so far.

Assertions cover the three levels that matter: the opening sequence (joins/start, record-equality on whole events), the scripted landmarks (bob's bridge jump, alice's goose hop, alice's exact win), and — the event-sourcing money shot — that **folding the observed history reproduces the expected final state**

(`{alice=63, bob=21}`, FINISHED, winner alice). Finally the test asserts clean shutdown: `close()` returns and the server thread actually dies.

What automation missed and live testing caught

The project's most instructive testing lesson is issue #7: 39 engine tests, a green E2E — and the **first live smoke game** froze the system. Two piped TUI clients blindly rolling every second (safe because out-of-turn rolls are rejected without effect) hit the well+prison mutual deadlock within one game. Unit tests verify the rules you wrote; free-running play explores the state space you didn't think to write rules about. The fix came with two new unit tests (`lastFreePlayerIsNeverTrapped`, `trapStillAppliesWhileAnotherPlayerIsFree`) — live testing finds, unit tests pin.

The same blind-roll harness became the **final verification** rig (PLAN.md Step 8): clean `mvn verify`, fresh cluster, then complete games through the real compose cluster — including one played start-to-win with a broker stopped the whole time, ISR healing verified afterwards, and a fresh client rebuilding the finished board by replay alone.

That last run produced its own lesson (ISSUES.md #8): the first broker-kill orchestration waited for `GameStarted`, slept, then killed the broker "mid-game" — but blind-roll games finish in ~90 seconds, and the game was already *won* before the kill fired; the "moves kept flowing" observation was vacuously true on a finished game. The fix: make the fault a **precondition** instead of an interruption — stop the broker first, then play the entire game. Timing-based fault injection against a fast workload silently tests nothing.

Testing patterns applied

- **Seams over mocks** — `DiceRoller`/`Clock` injection; a queue and a fixed clock replace any mocking framework.
- **Pin the policy** — behaviors that are *choices* (unknown-field tolerance, tombstone nulls, the overfull-cell fallback, the dead-

lock waiver) each have a test whose only job is to fail if the choice silently changes.

- **Test through the public surface** — the E2E drives topics, not methods; renderer tests read strings, not internals.
- **Deterministic by construction** — react to events rather than sleep; scripted inputs rather than seeded randomness.
- **ANSI-strip before assert** — layout tests match plain text, so color changes don't break them (and one test asserts colors exist and strip clean).

Testing anti-patterns avoided

- **Sleep-based synchronization** — nowhere in the suite; the one deadline is a backstop with diagnostics, not a wait.
- **Mocking the infrastructure you're trying to learn** — the broker is real where the broker matters.
- **Asserting on incidental detail** — whole-record equality where the contract is the record; `contains` on rendered text where layout is the contract.
- **Green-suite complacency** — the live smoke layer exists precisely because the unit layer can only check known rules (and #7 proved the point).

Issues (from ISSUES.md)

#4 — Testcontainers vs. Docker daemon 29 (fixed via `api.version` system property in failsafe). **#7** — the deadlock, found live. **#8** — the raced broker-kill test, fixed by inverting fault injection into a precondition. Also, review-cycle test bugs worth remembering: one renderer test initially asserted a goose marker on square 55 (not a goose square) — the *test* was wrong, the renderer right; a smoke-game script once computed `$(date +%s)` per client, putting the two players in different games.

09 — Patterns Applied & Anti-Patterns Avoided: the Catalog

The consolidated reference. Each entry names where the pattern lives and which chapter explains the reasoning. The through-line: **most entries are Java 21 language features or Kafka primitives used as intended** — the project's bet is that modern language design has absorbed many patterns that once needed hand-rolling.

Architectural patterns

Pattern	Where	Chapter
Event Sourcing	`game.events` as sole source of truth; all state a fold; replay everywhere	01
CQRS (miniature)	Commands and events as separate topics *and* separate sealed types; `GameView` as the read-side projection	01, 05
Single Writer	Only the server writes `game.events`; per-game single writer via gameId keying	01
Functional core, imperative shell	Pure `engine` inside the Kafka-facing `server`; pure `BoardRenderer` inside stdio `Main`	03, 04, 06
Ports & adapters	`GameListener` as the UI port; `client-core` knows no UI	05
State machine	`Phase` enum + exhaustive switches; invalid transitions are rejections	03
Outbox-style ordering	Produce-confirm *then* offset-commit — never acknowledge input before output is durable	04

Design & language patterns (Java 21 as the pattern language)

Pattern	Realization	Chapter
Algebraic data types	`sealed interface` + `record` per case for `Command`/`Event`; exhaustive `switch` with **no `default`** so protocol growth breaks every fold/renderer at compile time	02
Parse, don't validate	All validation in compact constructors; an invalid message cannot exist as an object, whatever its origin	02
Make illegal states unrepresentable	`Optional` components for absence; `Phase` instead of boolean flags; deep-immutable records (`List.copyOf`/`Map.copyOf` in compact constructors)	02, 03
Strategy via functional interface	`DiceRoller` — the single seam that replaces a mocking framework; `Clock` likewise	03, 08
Static factory	`GameClient.connect()` — prevents this-escape, names the operation	05
Withers for derived records	Explicit `withPosition`/`withStuck`/… copy methods (Java has no `with` expressions through 25)	03
Value-based equality as a test tool	Whole-event `assertEquals` in the E2E works because messages are records	08

Concurrency patterns

Pattern	Where	Chapter
Thread confinement	Server: one thread owns consumers, producer, and the state map — zero locks; client event loop likewise	04, 05
Signal-don't-touch shutdown	`volatile` flag + `consumer.wakeup()` (the one thread-safe consumer method) + bounded latch/join; the opener closes its own resources	04, 05
Virtual threads for I/O loops	Client event loop; server-on-virtual-thread in the E2E	05, 08
Immutable snapshot publication	`volatile GameView view` — safe unsynchronized reads because the value is immutable	05

Kafka patterns

Pattern	Where	Chapter
Keyed partitioning for per-entity order	Everything keyed by gameId	01
At-least-once with explicit commit ordering	flush + confirm futures before `commitSync`; auto-commit off	04
Idempotent producer + `acks=all` + min-ISR 2	Server producer / topic config	04, 07
Replay via manual assign	No group for the server's startup replay; throwaway groups + `earliest` for clients	04, 05
Poison-pill seek-past	Catch `RecordDeserializationException`, log, `seek(offset+1)`	04
Tombstone pass-through	Serde's null-in/null-out; explicit null handling downstream	02

Anti-patterns avoided — and where the temptation was real

Security & robustness:

- **Jackson polymorphic default typing** (gadget-chain RCE vector) → closed `@JsonSubTypes` on sealed types; the wire can name 11 record types, ever.

- **Unbounded/unvalidated input** → 10 KiB payload cap before parsing; `[A-Za-z0-9_-]` charsets killing log/ANSI/shell injection at the boundary.
- **Client-trusted randomness or state** → dice only server-side, from `SecureRandom`.
- **Crash-on-bad-record consumer loops** → poison pills skipped, never fatal.

Correctness:

- **Dual writes** → local state fed only by the exact events produced.
- **Silent delivery semantics** → the at-least-once window is documented at the code that creates it; auto-commit (which would silently give at-most-once here) explicitly disabled.
- **Duplicated business logic** → rules live once in `decide`; the fold trusts the log. The one *deliberate* duplication (client fold) is analyzed, bounded by shared protocol tests, and documented — the judgment call between harmful duplication and false sharing, made explicitly.
- **Unbounded rule loops** → goose-chain termination proven; zero-step rolls rejected at the `Board.resolve` boundary.
- **Exception-driven control flow** → rejection is data (empty list), not a throw.

Concurrency:

- **Shared mutable state under optimistic assumptions** → confinement, not locks-added-later.
- **this-escape from constructors** → `connect()` factory starts the thread after construction.
- **Swallowed `InterruptedException`** → always re-interrupt, then propagate.
- **`volatile` read-modify-write** → volatile only for flags/snapshot refs.

Hygiene:

- **Null-returning APIs** → `Optional`/empty collections; the two tombstone nulls are the single commented contract exception.

- **Lost exception causes** → every wrap passes the cause; poison-pill messages use `toString()` because `getMessage()` can be null.
- **Locale-dependent machine output** → `Locale.ROOT` pinned in the renderer.
- **Raw control bytes in source** → ANSI escapes only as `\u001B` literals (a hazard the project hit twice — once in generated Java, once in these very docs, both times introduced by tooling and caught by inspection).
- **Version drift** → every library and plugin pinned via the parent POM.
- **Sleep-based synchronization in tests and orchestration** → event-reactive test driver; health-checked compose startup; and after issue #8, fault injection as precondition instead of timing.

Where to read the histories

- `DECISIONS.md` — every non-obvious call, grouped by step, with reasoning and revisit-conditions.
- `ISSUES.md` — all 8 issues: symptom, failed attempts, the actual fix, and the transferable lesson.